Asynchronous Sessions with Input Races

Paola Giannini*

joint work with Ilaria Castellani‡ and Mariangiola Dezani†

∗ DiSSTE, Università del Piemonte Orientale
‡ INRIA Sophia Antipolis, France
† Dipartimento di Informatica, Università di Torino

PLACES 2022 Apr 3, 2022

Overall goal

Define a permissive type system for multiparty asynchronous sessions,

- allowing syntactic input races in processes (= choices of inputs from different senders)
- provided they do not hinder lock-freedom.

Global Types

Global types ensure global properties of multiparty sessions

Classical Structure of Typing

global type
$\downarrow$ projection
local/session types
$\downarrow$ type checking
processes

- Projectability is the main tool to enforce good properties,
- Projectability imposes restrictions on the choices that can be used in a protocol.
- It is difficult to accommodate protocols with input races!

Example 1: A "fake" input race

$\boxed{\texttt{Alice}}$ $\texttt{Bob!arr};\texttt{Bob?arr};(\,\texttt{Bob!tellC};\texttt{Carol!wArr}$
$\oplus$
$\texttt{Bob!yTellC}$
$)$

$\boxed{\texttt{Bob}}$ $\texttt{Alice!arr};\texttt{Alice?arr};(\,\texttt{Alice?tellC}\,,$
$+$
$\texttt{Alice?yTellC};\texttt{Carol!wArr}$
$)$

$\boxed{\texttt{Carol}}$ $(\,\texttt{Alice?wArr} + \texttt{Bob?wArr}\,)$

Example 2: A "confluent" input race

$\boxed{\texttt{Alice}}$ $\texttt{Carol!arr;}\ldots$

$\boxed{\texttt{Bob}}$ $\texttt{Carol!arr;}\ldots$

$\boxed{\texttt{Carol}}$ $(\texttt{Alice?arr;Bob?arr;}\ldots + \texttt{Bob?arr;Alice?arr;}\ldots)$

Objective and Approach

Define global types for multiparty sessions such that

- typable multiparty sessions enjoy lock freedom
- protocols such as the ones of the previous examples be typable

Approach

- define more expressive global types: split outputs and inputs
- bypass projection by matching directly the global type with the multiparty session imposing restrictions on the global type

Processes

$$P ::=_\rho \oplus_{i \in I} \mathsf{p}_i!\ell_i; P_i \mid \Sigma_{i \in I} \mathsf{p}_i?\ell_i; P_i \mid \mathbf{0}$$

coinductive definition, processes are regular terms

- $\oplus_{i \in I} \mathsf{p}_i!\ell_i; P_i$ = internal choice/output
- $\Sigma_{i \in I} \mathsf{p}_i?\ell_i; P_i$ = external choice/input
- $\mathbf{0}$ = inactive process

Example

$$P = \mathsf{q}\,!\,\ell_1; \mathsf{q}\,?\,\ell_2; P \qquad Q = \mathsf{p}\,!\,\ell_2; \mathsf{p}\,?\,\ell_1; Q$$

Multiparty Sessions

Network

$$\mathbb{N} ::= \mathsf{p}_1[\![P_1]\!] \parallel \ldots \parallel \mathsf{p}_n[\![P_n]\!]$$

where $\mathsf{p}_i \neq \mathsf{p}_j$ if $i \neq j$. $\mathsf{Players}(\mathbb{N}) = \{\mathsf{p} \mid \mathsf{p}[\![P]\!] \in \mathbb{N} \ \wedge \ P \neq \mathbf{0}\}$

Queue

$$\mathcal{M} ::= \emptyset \mid \langle \mathsf{p}, \ell, \mathsf{q} \rangle \cdot \mathcal{M}$$

where $\langle \mathsf{p}, \ell, \mathsf{q} \rangle$ is a message from p to q with label ℓ

we consider queues modulo an equivalence $\equiv$

$$\mathcal{M}_1 \cdot \langle \mathsf{p}, \ell_1, \mathsf{q} \rangle \cdot \langle \mathsf{r}, \ell_2, \mathsf{s} \rangle \cdot \mathcal{M}_2 \equiv \mathcal{M}_1 \cdot \langle \mathsf{r}, \ell_2, \mathsf{s} \rangle \cdot \langle \mathsf{p}, \ell_1, \mathsf{q} \rangle \cdot \mathcal{M}_2 \quad \text{if } \mathsf{p} \neq \mathsf{r} \text{ or } \mathsf{q} \neq \mathsf{s}$$

Session

network + queue $\qquad \mathbb{N} \parallel \mathcal{M}$

LTS for Asynchronous Sessions

Rules

[Send]

$$\mathsf{p}[\![\, \oplus_{i\in I}\mathsf{q}_i!\ell_i;\, P_i \,]\!] \parallel \mathbb{N} \parallel \mathcal{M} \xrightarrow{\mathsf{p}!\mathsf{q}_h.\ell_h} \mathsf{p}[\![\, P_h \,]\!] \parallel \mathbb{N} \parallel \mathcal{M}\cdot\langle \mathsf{p}, \ell_h, \mathsf{q}_h\rangle \quad h \in I$$

[Rcv]

$$\mathsf{q}[\![\, \Sigma_{j\in J}\mathsf{p}_j?\ell_j;\, Q_j \,]\!] \parallel \mathbb{N} \parallel \langle \mathsf{p}_h, \ell_h, \mathsf{q}\rangle\cdot\mathcal{M} \xrightarrow{\mathsf{q}?\mathsf{p}_h.\ell_h} \mathsf{q}[\![\, Q_h \,]\!] \parallel \mathbb{N} \parallel \mathcal{M} \quad h \in J$$

Asynchronous Global Types

$$\mathsf{G} ::=_\rho \mathsf{p}!\{\mathsf{q}_i.\ell_i;\mathsf{G}_i\}_{i\in I} \mid \mathsf{p}?\{\mathsf{q}_i.\ell_i;\mathsf{G}_i\}_{i\in I} \mid \mathsf{End}$$

global types are regular terms

- $\mathsf{p}!\{\mathsf{q}_i.\ell_i;\mathsf{G}_i\}_{i\in I}$ = output choice (p sends to q_i a label ℓ_i)
- $\mathsf{p}?\{\mathsf{q}_i.\ell_i;\mathsf{G}_i\}_{i\in I}$ = input choice (p receives from q_i label ℓ_i)
- End = termination

Example

$$\mathsf{G} = \mathsf{p}!\mathsf{q}.\ell_1;\mathsf{q}!\mathsf{p}.\ell_2;\mathsf{p}?\mathsf{q}.\ell_2;\mathsf{q}?\mathsf{p}.\ell_1;\mathsf{G}$$

Typing (1)

Typing judgement

$$\mathsf{G} \vdash \mathbb{N} \parallel \mathcal{M}$$

$$[\text{End}]\ \frac{}{\mathsf{End} \vdash \mathsf{p}[\![\, \mathbf{0} \,]\!] \parallel \emptyset}$$

$$[\text{Out}]\ \frac{\mathsf{G}_i \vdash \mathsf{p}[\![\, P_i \,]\!] \parallel \mathbb{N} \parallel \mathcal{M} \cdot \langle \mathsf{p}, \ell_i, \mathsf{q}_i \rangle \quad \forall i \in I}{\mathsf{p}!\{\mathsf{q}_i.\ell_i; \mathsf{G}_i\}_{i \in I} \vdash \mathsf{p}[\![\, \bigoplus_{i \in I} \mathsf{q}_i!\ell_i; P_i \,]\!] \parallel \mathbb{N} \parallel \mathcal{M}} \quad (*)$$

where the side condition $(*)$ is

$$\mathsf{Players}(\mathsf{G}_i) \setminus \{\mathsf{p}\} = \mathsf{Players}(\mathbb{N}) \quad \forall i \in I$$

where

$$\mathsf{Players}(\mathsf{End}) = \emptyset$$

$$\mathsf{Players}(\mathsf{p}!\{\mathsf{q}_i.\ell_i; \mathsf{G}_i\}_{i \in I}) = \mathsf{Players}(\mathsf{p}?\{\mathsf{q}_i.\ell_i; \mathsf{G}_i\}_{i \in I}) = \{\mathsf{p}\} \cup \bigcup_{i \in I} \mathsf{Players}(\mathsf{G}_i)$$

Typing Rules: the Difficult Rule

$$[\mathsf{In}]\ \frac{\mathsf{G}_j \vdash \mathsf{p}[\![P_j]\!] \parallel \mathbb{N} \parallel \mathcal{M}_j \quad \iota(\mathsf{G}_j, \mathbb{M}) \quad \forall j \in J}{\mathsf{p}?\{\mathsf{q}_i.\ell_i; \mathsf{G}_i\}_{i\in I} \vdash \mathsf{p}[\![\sum_{h\in H} \mathsf{q}_h?\ell_h; P_h]\!] \parallel \mathbb{N} \parallel \mathcal{M}} \quad \begin{array}{l} \overset{(*)}{J} = \mathsf{rm}(\{\mathsf{m}_i\}_{i\in I}, \mathcal{M}) \\ \quad = \mathsf{rm}(\{\mathsf{m}_h\}_{h\in H}, \mathcal{M}) \\ \quad \neq \emptyset \end{array}$$

- Let $\mathsf{m}_k = \langle \mathsf{q}_k, \ell_k, \mathsf{p} \rangle$.
- The indexes of ready messages of a set of messages $\{\mathsf{m}_i\}_{i\in I}$ w.r.t a queue $\mathcal{M}$ is
$$\mathsf{rm}(\{\mathsf{m}_i\}_{i\in I}, \mathcal{M}) = \{i \in I \mid \mathcal{M} \equiv \mathsf{m}_i \cdot \mathcal{M}_i\}.$$
- G is inactive for $\mathbb{M}$
$$\iota(\mathsf{End}, \mathbb{M}) \qquad \iota(\mathsf{p}?\{\mathsf{q}_i.\ell_i; \mathsf{G}_i\}_{i\in I}, \mathbb{M}) \quad \text{if } \iota(\mathsf{G}_i, \mathbb{M}) \ \ \forall i \in I$$
$$\iota(\mathsf{p}!\{\mathsf{q}_i.\ell_i; \mathsf{G}_i\}_{i\in I}, \mathbb{M}) \quad \text{if } \langle \mathsf{p}, \ell_i, \mathsf{q}_i \rangle \notin \mathbb{M} \text{ and } \iota(\mathsf{G}_i, \mathbb{M}) \ \ \forall i \in I$$

Here $\mathbb{M} = \{\langle \mathsf{q}_l, \ell_l, \mathsf{p} \rangle \mid l \in (I \cup H) \setminus J \ \&\ \mathsf{q}_l \neq \mathsf{q}_j \ \forall j \in J\}$.

We need $\iota(\mathsf{G}, \mathbb{M})$!

Consider the multiparty session $\mathbb{N} \parallel \langle \mathsf{p}, \ell, \mathsf{r} \rangle$ where

$$\mathbb{N} = \mathsf{q}[\![\, \mathsf{r}!\ell' \,]\!] \parallel \mathsf{r}[\![\, \mathsf{p}?\ell; \mathsf{q}?\ell' + \mathsf{q}?\ell'; \mathsf{p}?\ell' \,]\!]$$

we expect $\mathbb{N} \parallel \langle \mathsf{p}, \ell, \mathsf{r} \rangle$ not to be typable, since it can reach a deadlock.

$$\begin{array}{rcl} \mathbb{N} \parallel \langle \mathsf{p}, \ell, \mathsf{r} \rangle & \xrightarrow{\mathsf{q}!\mathsf{r}.\ell'} & \mathsf{r}[\![\, \mathsf{p}?\ell; \mathsf{q}?\ell' + \mathsf{q}?\ell'; \mathsf{p}?\ell' \,]\!] \parallel \langle \mathsf{p}, \ell, \mathsf{r} \rangle \cdot \langle \mathsf{q}, \ell', \mathsf{r} \rangle \\ & \xrightarrow{\mathsf{r}?\mathsf{q}.\ell'} & \mathsf{r}[\![\, \mathsf{p}?\ell' \,]\!] \parallel \langle \mathsf{p}, \ell, \mathsf{r} \rangle \end{array}$$

Let $\mathsf{G} = \mathsf{r}?\{\mathsf{p}.\ell; \mathsf{q}!\mathsf{r}.\ell'; \mathsf{r}?\mathsf{q}.\ell' \,,\, \mathsf{q}.\ell'; \mathsf{q}!\mathsf{r}.\ell'; \mathsf{r}?\mathsf{p}.\ell'\}$.

$$\dfrac{\dfrac{\dfrac{\mathsf{End} \vdash \mathsf{r}[\![\, \mathbf{0} \,]\!] \parallel \emptyset}{\mathsf{r}?\mathsf{q}.\ell' \vdash \mathsf{r}[\![\, \mathsf{q}?\ell' \,]\!] \parallel \langle \mathsf{q}, \ell', \mathsf{r} \rangle}\,[\mathsf{In}]}{\mathsf{q}!\mathsf{r}.\ell'; \mathsf{r}?\mathsf{q}.\ell' \vdash \mathsf{q}[\![\, \mathsf{r}!\ell' \,]\!] \parallel \mathsf{r}[\![\, \mathsf{q}?\ell' \,]\!] \parallel \emptyset}\,[\mathsf{Out}]}{\mathsf{r}?\{\mathsf{p}.\ell; \mathsf{q}!\mathsf{r}.\ell'; \mathsf{r}?\mathsf{q}.\ell' \,,\, \mathsf{q}.\ell'; \mathsf{q}!\mathsf{r}.\ell'; \mathsf{r}?\mathsf{p}.\ell'\} \vdash \mathbb{N} \parallel \langle \mathsf{p}, \ell, \mathsf{r} \rangle}\,[\mathsf{In}]$$

Checking $\iota(\mathsf{q}!\mathsf{r}.\ell'; \mathsf{r}?\mathsf{p}.\ell', \{\langle \mathsf{q}, \ell', \mathsf{r} \rangle\})$ we fail to apply [In]

Properties

We have an LTS for type configurations: $\mathsf{G} \parallel \mathcal{M} \xrightarrow{\beta} \mathsf{G}' \parallel \mathcal{M}'$
where $\beta \in \{\mathsf{p!q}.\ell, \mathsf{p?q}.\ell \mid \mathsf{p}, \mathsf{q} \in \mathsf{Part}, \ell \in \mathsf{Lab}\}$,

If $\mathsf{G} \vdash \mathbb{N} \parallel \mathcal{M}$ and G is bounded (each $\mathsf{p} \in \mathsf{Players}(\mathsf{G})$ occurs at bounded depth in all paths of G),then we can prove the properties

- Session Fidelity
- Subject Reduction
- Lock Freedom

However

- The type system is undecidable.

A decidable restriction can be defined giving a (standard) inductive definition.

Related Work

- Our global types extend those of [1] by allowing input races.
- In [2] and [3] sessions are synchronous.
- None of the papers below can type the session of Example 1.
- Example 1 omitting the initial exchange between Alice and Bob can be typed in [2] and [4] but not in [3]
- Example 2 is typed in [3] but not in [2] and [4].

---

[1] F. Dagnino, M. D.-C. & P.G. (2021): Deconfined Global Types for Asynchronous Sessions.
[2] R. van Glabbeek, P. Höffner & R. Horne (2021): Assuming Just Enough Fairness to make Session Types Complete for Lock-freedom.
[3] S. Jongmans & N. Yoshida (2020): ExploringType-Level Bisimilarity towards More Expressive Multiparty Session Types.
[4] R. Majumdar, M. Mukund, F. Stutz & D. Zufferey (2021): Generalising Projection in Asynchronous Multiparty Session Types.

Conclusions and Future Work

- We defined a type system for multiparty asynchronous sessions, allowing syntactic input races and ensuring lock-freedom.
- There are several directions for future work:
  - weaken the predicate $\iota(\mathsf{G}, \mathbb{M})$ taking into account the order of messages sent by G
  - strengthen our typing in order to forbid orphan messages
  - weaken our typing in order to allow optional participants in the branches of choices, following some previous work on "connecting communications"
  - and more....

obrigado
Dank U
Merci
mahalo
Köszi
спасибо
Grazie
Thank you
mauruuru
Takk
Gracias
Dziękuję
Děkuju
danke
Kiitos